

Exercícios OAC

RISC-V Uniciclo

Instruções diferentes utilizam blocos de hardware diferentes na implementação uniciclo. Considere as instruções:

- a. `add rd, rs1, rs2` $xregs[rd] = xregs[rs1] + xregs[rs2]$
- b. `lw rd, offs(rs1)` $xregs[rd] = mem[xregs[rs1] + offs]$

- 1) Quais os valores dos sinais de controle gerados pela parte de controle uniciclo para estas instruções ?
- 2) Quais recursos (blocos) do *hardware* são utilizados nestas instruções ?
- 3) Quais blocos produzem saídas que não são utilizadas para executar estas instruções ?

A tabela abaixo indica dois conjuntos alternativos de latências para os blocos do RISC-V uniciclo.

	Mem Inst	Somador	Mux	ULA	XREGS	Mem D.	Controle
a	400 ps	100 ps	30 ps	120 ps	200 ps	350 ps	100 ps
b	500 ps	150 ps	100 ps	180 ps	220 ps	350 ps	65 ps

- 4) Qual o caminho crítico para a instrução AND ?
- 5) Qual o caminho crítico para a instrução LW ?
- 6) Qual o caminho crítico para a instrução BEQ ?
- 7) Se fosse possível construir um processador uniciclo ideal onde cada instrução tivesse um atraso igual ao seu caminho crítico, o quanto mais rápido seria esse processador do que o uniciclo normal, para os casos a e b acima, considerando a seguinte distribuição de instruções:
 - a. Log-Arith: 50%; LW: 25%; SW: 15%; BEQ: 10%;
- 8) Considere que a adição de uma unidade de multiplicação na implementação **a** aumenta o atraso da ULA para 300 ps, mas reduz o número de instruções do item anterior em 5%.
 - a. Como fica o período do relógio para o uniciclo com esta alteração ?
 - b. Como fica o desempenho comparando as duas soluções para o programa do item 7?

A implementação básica do RISC-V uniciclo pode apenas implementar algumas instruções. Novas instruções podem ser adicionadas a uma ISA existente, mas a decisão de fazer ou não depende, entre outras coisas, do custo e da complexidade que tal adição introduz no caminho de dados e no controle do processador. Os três primeiros problemas deste exercício referem-se a estas novas instruções:

- a. `add3 rd, rs1, rs2, rs3` $xregs[rd] = xregs[rs1] + xregs[rs2] + xregs[rs3]$
- b. `addz rd, rs1, rs2` $xregs[rd] = xregs[rs1] + xregs[rs2]$ se a operação *anterior* produziu zero na saída da ULA (soma se zero).
- 9) Quais blocos do hardware podem ser utilizados para essas instruções ? Quais blocos (se houver) devem ser acrescentados para viabilizar sua execução ?
- 10) Quais sinais de controle devem ser acrescentados / modificados (se necessário) para suportar a execução destas instruções ?

Uma falha comum na fabricação de circuitos integrados é ligar um sinal permanentemente à alimentação (VDD) ou ao terra (GND). Se um sinal está ligado ao terra, ele produz sempre o valor lógico zero. Este problema é conhecido como falha “stuck-at-0”.

- 11) Quais instruções não executariam corretamente se o sinal MemtoReg no RISC-V uniciclo estivesse *stuck-at-0* ?
- 12) Quais instruções não executariam corretamente se o sinal ALUSrc no RISC-V uniciclo estivesse *stuck-at-0* ?

RISC-V Multiciclo

- 1) Considerando a arquitetura multiciclo anexa, indique os valores dos sinais de controle do processador para a execução das instruções:
 - a. `beq rs1, rs2, LABEL`
 - b. `sw rs2, offs(rs1)`
 - c. `jalr rd, offs`
- 2) No RISC-V multiciclo simplificado ilustrado a seguir várias instruções importantes estão ausentes. Mantendo a compatibilidade do código em linguagem de máquina com o RISC-V original, redesenhe o caminho de dados multiciclo, redefina os sinais de controle e a máquina de estados (se necessário), de modo a implementar (além das instruções já existentes) as instruções de salto condicional:
 - a. `bne rs1, rs2, label`
 - b. `blt rs1, rs2, label`
 - c. `bge rs1, rs2, label`

obs: o sinal zero da ULA indica se o valor na saída da ULA é zero (32 bits). Pode, portanto, ser utilizado para identificar se os operandos são iguais ou diferentes. Além disso, o bit de sinal do resultado (bit 31 da saída da ULA) pode ser utilizado para identificar as relações $<$ e $\geq$ entre os operandos da ULA.
- 3) Considerando apenas os seguintes tempos de atraso do bloco operativo de uma CPU RISC-V:
 - Operação Lógica com a ULA (and,or): 200ps
 - Operação Aritmética com a ULA (add,sub,slt): 250ps
 - Leitura do Banco de Registradores: 100ps
 - Escrita no Banco de Registradores: 150ps
 - Leitura da memória: 400ps
 - Escrita na memória: 600ps

Com relação ao seguinte procedimento (PROC) em assembly, chamado com argumento `a0=10`:

```
PROC:  add t0, zero, zero
        add a1, zero, zero
        lw  t4, 0(gp)           # t4 = 1
```

```

        lui t1, 0x800000
LOOP:   and  t2, a0, t4
        beq  t2, zero, ELSE
        add  a1, t0, zero
ELSE:   beq  t1, t4, FIM
        addi t0, t0, 1
        slli t4, t4, 1
        j    LOOP
FIM:    add  a0, a1, zero
        jalr x0, ra, 0

```

Responda as questões abaixo:

- Quantas instruções esse código executa ?
- Qual informação esta função retorna ?
- Para uma implementação uniciclo, qual será a maior frequência de clock utilizável? Qual o tempo de execução deste trecho de código neste caso?
- Para uma implementação multiciclo, qual será a maior frequência de clock utilizável? Qual o tempo de execução deste trecho de código neste caso? Qual o CPI deste código, considerando LW: 5 ciclos; LogArit: 4 ciclos; Jumps e branches: 3 ciclos;
- Para uma implementação com pipeline ideal, isto é, ignorando conflitos, *stalls* e latência, qual será a maior frequência de clock utilizável? Qual o tempo de execução deste trecho de código neste caso?
- Considerando o pipeline de 5 estágios visto em aula, onde o beq sem previsão é avaliado no 4º ciclo e o jump necessita 3 ciclos. Identifique e classifique os conflitos que ocorrem neste procedimento. Como eles podem ser resolvidos sem alteração no programa?
- Qual o tempo de execução, se todos os conflitos forem tratados apenas com inserção de bolhas? Considere que os registradores só podem ser lidos após sua escrita ser completada no banco de registradores.
- Qual o tempo de execução, se os conflitos forem tratados eficientemente com adiantamento e/ou bolhas? Defina o tipo de previsão mais adequado ao beq e ao bne neste programa. Considere que os registradores a serem escritos podem ser lidos no mesmo ciclo que serão escritos e os branches avaliados no 2º ciclo.

RISC-V Pipeline

Neste exercício, examinamos como o *pipeline* afeta o período do relógio do processador. Assume-se que os estágios de *pipeline* tem as seguintes latências:

Fetch	Decode	Execute	Memory	WB
250 ps	350 ps	150 ps	300 ps	200 ps

o código utilizado neste exemplo é distribuído entre as seguintes instruções:

- a. Log-Arith: 45% b. Jump/Branch: 20% c. Load: 20% d. Store: 15%

- Qual é o ciclo de relógio das versões *pipeline* e uniciclo ?
- Qual é a latência total da instrução *load* nas duas versões do processador ?
- Se pudéssemos dividir um estágio do *pipeline* em dois, cada um com metade da latência do original, qual estágio você dividiria e qual seria o novo período de relógio ?

4. Qual o percentual de utilização da memória de dados ?
5. Qual é o mínimo número de ciclos para executar completamente n instruções em uma CPU com um *pipeline* de k estágios ? Justifique sua fórmula.
6. Assuma que x11 e x12 estão iniciados com valores 11 e 22, respectivamente. Suponha que o código abaixo execute em um *pipeline* sem tratamento de conflitos (i.e., o programador é responsável pela inserção de *nops* quando necessário). Qual o valor final dos registradores x13 e x14 ?

```
addi x11, x12, 5
add  x13, x11, x12
addi x14, x11, 15
```

7. Assuma que x11 e x12 estão iniciados com valores 11 e 22, respectivamente. Suponha que o código abaixo execute em um *pipeline* sem tratamento de conflitos (i.e., o programador é responsável pela inserção de *nops* quando necessário). Qual o valor final do registrador x15, supondo adiantamento de dados apenas na leitura do banco de registradores, ou seja, o dado escrito num registrador é lido no mesmo ciclo.

```
addi x11, x12, 5
add  x13, x11, x12
addi x14, x11, 15
add  x15, x11, x11
```

8. Acrescente instruções NOP no código abaixo para eliminar os conflitos de dados, supondo que não haja nenhum tipo de adiantamento de dados.

```
addi x11, x12, 5
add  x13, x11, x12
addi x14, x11, 15
add  x15, x13, x12
```

Considere uma versão do *pipeline* que não trata conflitos de dados. Suponha que (depois de otimizado) um programa típico com n instruções requer em média $0,4 * n$ NOPs para eliminar os conflitos.

9. Supor que o ciclo de relógio deste pipeline sem adiantamento é 250 ps. Considere que acrescentando o *hardware* para adiantamento de dados reduza o número médio de NOPs de $0,4 * n$ para $0,05 * n$, mas aumente o período do relógio para 300 ps. Qual a aceleração obtida com o adiantamento de dados ?
10. Diferentes programas irão requerer quantidades de NOPs distintas. Considerando que o programa sem *pipeline* requer $0,4 * n$ NOPs, a partir de qual percentual de NOPs o *pipeline* com adiantamento de dados passa a ser mais lento que o *pipeline* sem adiantamento, pressupondo o mesmo aumento no período do relógio ? 11

Se alterarmos as instruções *load/store* de forma que elas não usem mais um dado imediato no cálculo do endereço de memória, ou seja, o endereço é determinado apenas pelo registrador base *rs1*, essas instruções não utilizarão mais a ULA em sua execução. Como consequência, os estágios MEM e EXEC podem ser sobrepostos, e o pipeline reduzido para apenas 4 estágios (IF / ID / MEMEXEC / WB).

11. Como a redução da profundidade do *pipeline* afeta o período do relógio ?
12. Como pode esta mudança melhorar o desempenho do *pipeline* ?
13. De que forma essa mudança pode degradar o desempenho do *pipeline* ?
14. Qual dos diagrama de *pipeline* abaixo melhor descreve a operação da unidade de detecção de conflitos no *pipeline* ? Por que ?

a.	lw	x11, 0(x12)	IF	ID	EX	MD	WB
	add	x13, x11, x14		IF	ID	EX	.. MD WB
	or	x15, x16, x17			IF	ID	.. EX MD WB
b.	lw	x11, 0(x12)	IF	ID	EX	MD	WB
	add	x13, x11, x14		IF	ID	.. EX MD WB	
	or	x15, x16, x17			IF	.. ID EX MD WB	

15. Considere a organização RISC-V *pipeline* anexa. Acrescente a lógica para realização da instrução BEQL (*Branch on Equal Likely*). Essa instrução funciona de forma similar a BEQ, com a diferença de que se o desvio NÃO é tomado, o *hardware* anula a instrução seguinte ao BEQL (introduz uma bolha). Se o desvio é tomado, a instrução seguinte ao BEQL é executada. Explique e ilustre na figura que modificação deve ser feita para permitir a execução dessa nova instrução

Hierarquia de memória

Considere o seguinte mix de instruções na execução de um programa P:

R-type	I-type (sem lw)	Load	Store	Branch	Jump
24%	28%	25%	10%	11%	2%

- 1) Qual fração de todas as instruções usa a memória de dados ?
- 2) Qual fração de todas as instruções usa a memória de instruções ?
- 3) Qual fração de todas as instruções utiliza a geração de imediatos ?
- 4) O que o bloco de geração de imediatos faz durante os ciclos em que sua saída não é utilizada ?

Considere uma *cache* com 16KiB de dados e blocos de 16 palavras de 32 bits. O endereço também é de 32 bits.

- 5) Qual o tamanho do campo de TAG ?
- 6) Quantas entradas tem essa cache ?
- 7) Calcule o número de bits necessário para implementar esta cache.
- 8) O quanto maior é a quantidade de bits necessária para implementar uma cache de 32 KiB de dados e blocos de 16 palavras de 32 bits ?

Uma *cache* diretamente mapeada com endereço e palavras de 32 bits tem a seguinte distribuição de campos do endereço:

TAG	INDEX	Offset
31 – 10	9 – 5	4 – 0

- 9) Qual o tamanho do bloco da *cache* (em palavras) ?
- 10) Quantos blocos essa *cache* tem ?
- 11) Qual a razão entre o total de bits necessários a implementação da cache com relação ao total de bits de dados ?

Considere a sequência de acessos à memória, onde os endereços são **endereços de palavras** (não bytes):

0x03, 0xb4, 0x2b, 0x02, 0xbf, 0x58, 0xbe, 0x0e, 0xb5, 0x2c, 0xba, 0xfd

- 12) Para cada uma destas referências, mostre o seu código binário, a tag e o índice para uma *cache* com mapeamento direto contendo 16 blocos de uma palavra cada. Indique para cada acesso se ocorre um acerto ou falha, supondo a *cache* inicialmente vazia.
- 13) Para cada uma destas referências, mostre o seu código binário, a tag, o índice e o offset para uma *cache* com mapeamento direto contendo 8 blocos de duas palavra cada. Indique para cada acesso se ocorre um acerto ou falha, supondo a *cache* inicialmente vazia.

Suponha uma aplicação que realiza a leitura de um vetor com 1024 palavras. São gerados os endereços de 0 a 1023 sequencialmente, uma única vez. Qual seria a taxa de falhas para as seguintes configurações de *cache* diretamente mapeada:

- 14) 256 entradas com blocos de uma palavra
- 15) 128 entradas com blocos de 2 palavras
- 16) 64 entradas com blocos de 4 palavras
- 17) Como se comporta a taxa de falhas em relação aos parâmetros da *cache* para este tipo de acesso ?

O tamanho do bloco da *cache* (B) pode afetar a taxa de falhas e a latência da falha (ou penalidade). Supondo uma máquina com uma CPI base de 1 e uma média de 1,35 referências (instrução e dados) por instrução, encontre o tamanho do bloco que minimiza a latência total da falha, dadas as seguintes taxas de falha para vários tamanhos de bloco.

8 : 4% 16 : 3% 32 : 2% 64 : 1,5 % 128 : 1%

- 18) Qual o tamanho ótimo do bloco para uma latência de falha de $20 \times B$ ciclos ?
- 19) Qual o tamanho ótimo do bloco para uma latência de falha de $20 + B$ ciclos ?
- 20) Qual o tamanho ótimo do bloco para uma latência de falha constante ?

Neste exercício, veremos as diferentes maneiras pelas quais a capacidade afeta o desempenho geral. Em geral, o tempo de acesso a *cache* é proporcional à capacidade. Suponha que os acessos à memória principal levam 70 ns e que 36% de todas as instruções acessam a memória de dados. A tabela a seguir mostra os dados dos caches L1 anexados a cada um de dois processadores, P1 e P2.

	L1 size	L1 miss rate	L1 hit time
P1	2 KiB	8,0 %	0,66 ns
P2	4 KiB	6,0 %	0,90 ns

- 21) Assumindo que o tempo de acerto determina o período do relógio, quais as frequências de relógio de P1 e P2 ?
- 22) Qual o tempo médio de acesso à memória para cada processador (independente do programa) ?
- 23) Assumindo um CPI de 1,0 sem falhas, qual o CPI total para P1 e P2 ? Qual é mais rápido ?

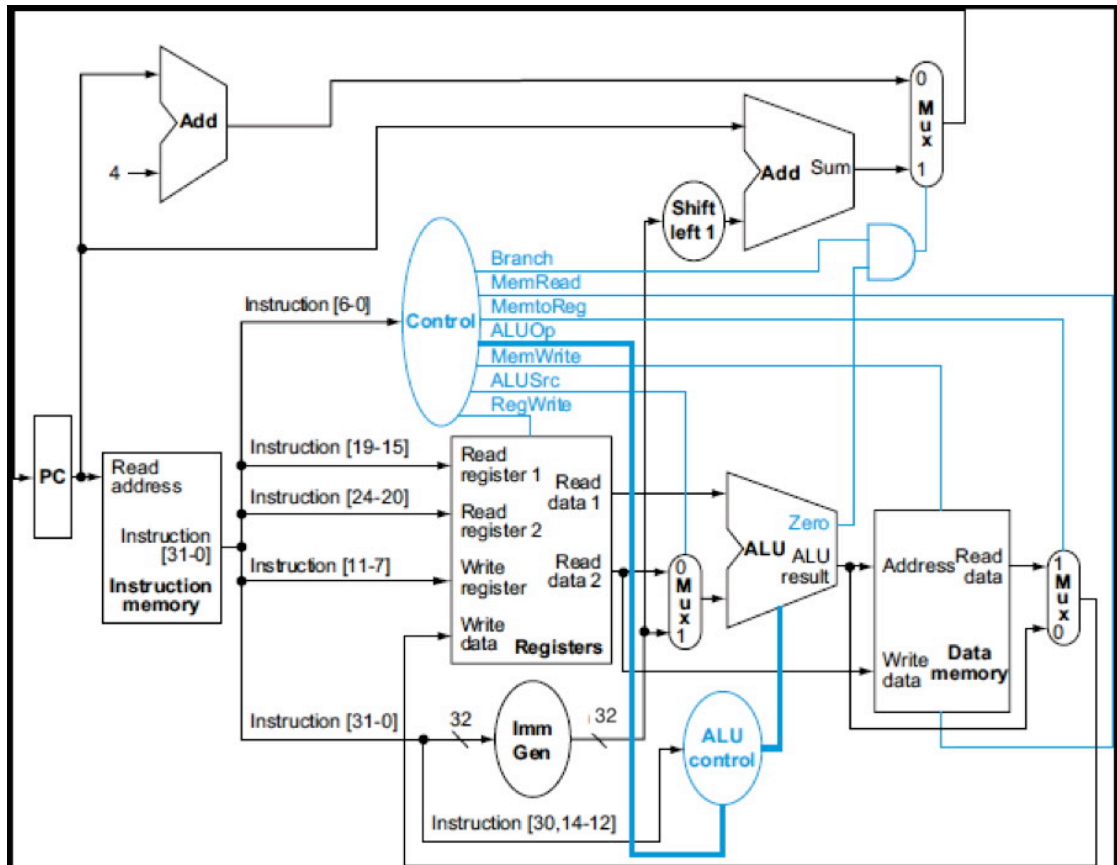
Suponha que uma taxa de falhas de *cache* de instruções para um programa seja de 1% e que uma taxa de falhas de *cache* de dados seja de 2,5%. O processador possui uma CPI de 1,0 para uma *cache* ideal e a penalidade de uma falha é de 80 ciclos.

- 24) Determine o CPI média levando em consideração apenas as falhas na *cache* de instruções.
- 25) Considerando um programa P com 30% acesso à memória de dados, calcule a variação de desempenho entre o processador com *cache* ideal e o processador com *cache* real.

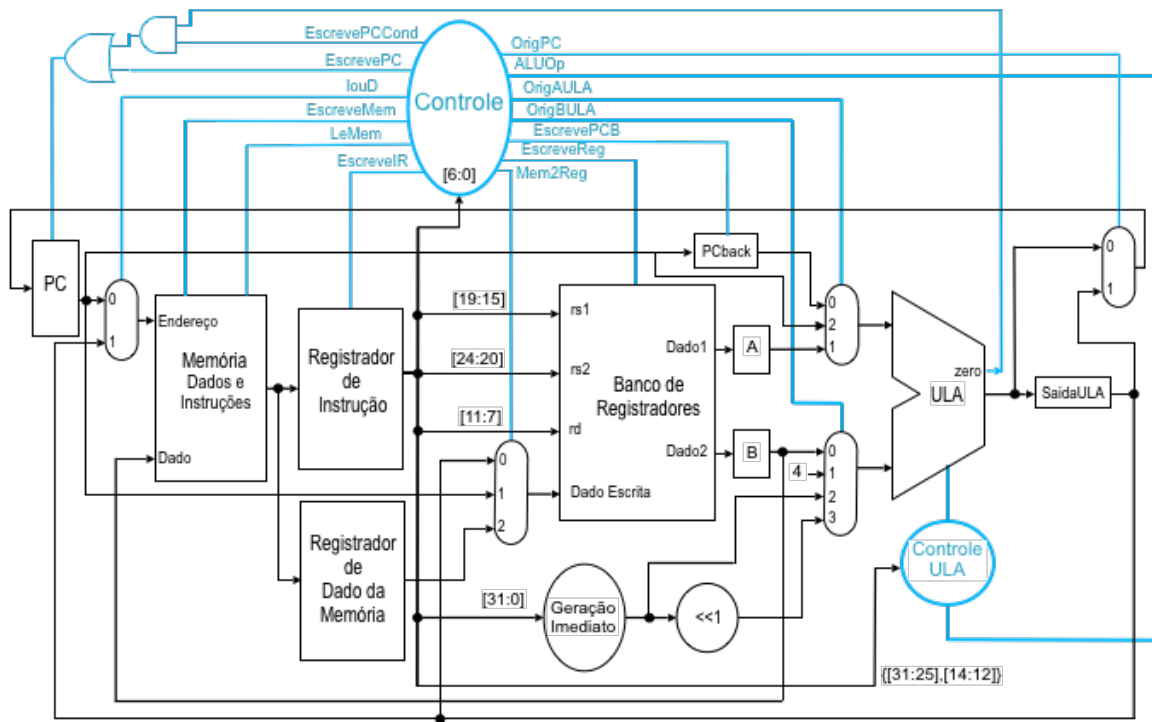
Considerando o mesmo processador, uma equipe de engenharia propôs uma otimização no sistema computacional. Aumentando a *cache* de dados, reduziu a taxa de falhas para 2,0%, mas aumentando a penalidade da falha para 100 ciclos. Outra equipe propôs a substituição do processador por um novo com o dobro da frequência.

- 26) Compare as duas otimizações com o projeto original e verifique os possíveis ganhos em desempenho.

RISC-V Uniciclo



RISC-V Multiciclo



RISC-V Pipeline

